Step 4 — A Test Suite That Catches a Bad Change Before It Ships (final) — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — What the API tests can't see
- Hint 2 — Proving the suite, and the judge, actually work

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — What the API tests can't see

The API tests fake the team, so they can't notice a **worse article**. That needs an eval suite (Doc13):

- a fixed list of topics, with what a good result looks like, written **before** you run anything;
- a code check where one exists (an exact `[source]` tag must appear; an unknown topic must give no article);
- a judge — a second model call with written rules — only where code can't check (e.g. "says how long panels take to pay back");
- one overall score.

Project 4's tests stay exactly as they are — plain scripts, run with `python`. Only the new tests in this project use pytest.

Doc13's Build Task files, inside the package:

- `evals/tasks.py` → 10 tasks, each with `"expected"`, `"rules"` or `"expect_no_article"`.
- `evals/judge.py` → Doc13's judge: `JudgeResult`, `JUDGE_PROMPT_VERSION`, `build_judge_prompt()`, `score_with_judge()` — with the OpenAI client made inside `call_model()`.
- `evals/run_eval.py` → `run_one_task()` calls `run_pipeline(task["topic"])` — the same door the API uses — then `run_eval_suite()` prints the score and saves `report.json`.

For the old tests: `assert len(lines) == 5` instead of printing the count; in the integration test, call `load_config()` at the top of the file and build the graph inside each test (pytest can't pass it in).

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — Proving the suite, and the judge, actually work

A suite you've never seen fail is one you don't know works. So break something on purpose:

1. run the suite and note the score;
2. make one prompt worse;
3. run it again — the score must drop, below a bar you chose **before** you started.

Pick a change the rest of the team can't repair. If Research's notes lose their `[source]` tags, no later agent can put them back.

And test the judge itself: give it answers where you already know the right verdict.

```
regression_check.py (Doc13):
    PASS_BAR = 0.8
    baseline = run_eval_suite()
    input("delete ', each ending with its [source] tag' from
           research.txt, then press Enter...")
    after = run_eval_suite()
    caught if score(baseline) >= PASS_BAR and score(after) < PASS_BAR
    print the tasks that newly failed
```

`tests/integration/test_judge.py` — a calibration set with `TITLE_RULE` and `SOURCE_RULE`: a titled article with a tag → `PASS`; `"I like pizza."` → `FAIL`; the same article **without** its tag → `FAIL`.

Then run the layers on purpose: the unit scripts and `pytest tests/api` (free) on every change; `python tests/integration/test_live_agents.py`, `pytest tests/integration/test_judge.py` and `run_eval` before a prompt change.

Full solution: Show me the solution